

1. Import packages and observe dataset

```
In [14]: import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
%matplotlib inline
```

```
In [15]: from sklearn import preprocessing
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import r2_score
```

2. Model building

```
In [16]: data = pd.read_csv(r'C:\Users\sande\OneDrive\Desktop\Naresh IT\data science and ai\
```

```
In [17]: data
```

```
Out[17]:
```

	mpg	cyl	displacement	hp	wt	acc	yr	origin	car_type	car_name
0	18.0	8	307.0	130	3504	12.0	70	1	0	chevrolet chevelle malibu
1	15.0	8	350.0	165	3693	11.5	70	1	0	buick skylark 320
2	18.0	8	318.0	150	3436	11.0	70	1	0	plymouth satellite
3	16.0	8	304.0	150	3433	12.0	70	1	0	amc rebel sst
4	17.0	8	302.0	140	3449	10.5	70	1	0	ford torino
...	...	...	...	...	...	...	...	...	...	...
393	27.0	4	140.0	86	2790	15.6	82	1	1	ford mustang gl
394	44.0	4	97.0	52	2130	24.6	82	2	1	vw pickup
395	32.0	4	135.0	84	2295	11.6	82	1	1	dodge rampage
396	28.0	4	120.0	79	2625	18.6	82	1	1	ford ranger
397	31.0	4	119.0	82	2720	19.4	82	1	1	chevy s-10

398 rows × 10 columns

```
In [18]: data.head()
```

```
Out[18]:
```

	mpg	cyl	dis	hp	wt	acc	yr	origin	car_type	car_name
0	18.0	8	307.0	130	3504	12.0	70	1	0	chevrolet chevelle malibu
1	15.0	8	350.0	165	3693	11.5	70	1	0	buick skylark 320
2	18.0	8	318.0	150	3436	11.0	70	1	0	plymouth satellite
3	16.0	8	304.0	150	3433	12.0	70	1	0	amc rebel sst
4	17.0	8	302.0	140	3449	10.5	70	1	0	ford torino

```
In [19]: data = data.drop(['car_name'], axis = 1)
```

```
In [20]: data['origin']= data['origin'].replace({1:'america', 2:'europe', 3:'asia'})
data = pd.get_dummies(data,columns = ['origin'],dtype=int)
data = data.replace('?', np.nan)
```

```
In [21]: data = data.apply(pd.to_numeric, errors='ignore')

numeric_cols = data.select_dtypes(include=[np.number]).columns
data[numeric_cols] = data[numeric_cols].apply(lambda x: x.fillna(x.median()))
```

C:\Users\sande\AppData\Local\Temp\ipykernel_8808\1419340433.py:1: FutureWarning: errors='ignore' is deprecated and will raise in a future version. Use to_numeric without passing `errors` and catch exceptions explicitly instead
data = data.apply(pd.to_numeric, errors='ignore')

```
In [22]: data.head()
```

```
Out[22]:
```

	mpg	cyl	dis	hp	wt	acc	yr	car_type	origin_america	origin_asia	origin_eur
0	18.0	8	307.0	130.0	3504	12.0	70	0	1	0	
1	15.0	8	350.0	165.0	3693	11.5	70	0	1	0	
2	18.0	8	318.0	150.0	3436	11.0	70	0	1	0	
3	16.0	8	304.0	150.0	3433	12.0	70	0	1	0	
4	17.0	8	302.0	140.0	3449	10.5	70	0	1	0	



```
In [23]: x = data.drop(['mpg'], axis = 1)
y = data[['mpg']]
```

```
In [26]: x_s = preprocessing.scale(x)
x_columns=x.columns
x_s=pd.DataFrame(x_s, columns= x_columns)

y_s=preprocessing.scale(y)
y_columns=y.columns
y_s=pd.DataFrame(y_s, columns= y_columns)
```

```
In [38]: x_train, x_test, y_train, y_test = train_test_split(x_s, y_s, test_size = 0.30, ra
x_train.shape
```

```
Out[38]: (278, 10)
```

2.a Simple Linear Model

```
In [39]: #Fit simple linear model and find coefficients
regression_model = LinearRegression()
regression_model.fit(X_train, y_train)

for idx, col_name in enumerate(X_train.columns):
    print('The coefficient for {} is {}'.format(col_name, regression_model.coef_[0]

intercept = regression_model.intercept_[0]
print('The intercept is {}'.format(intercept))
```

```
The coefficient for cyl is 0.32102238569161057
The coefficient for disp is 0.324834309184838
The coefficient for hp is -0.22916950059437718
The coefficient for wt is -0.7112101905072288
The coefficient for acc is 0.014713682764190883
The coefficient for yr is 0.3755811949510743
The coefficient for car_type is 0.3814769484233099
The coefficient for origin_america is -0.07472247547584182
The coefficient for origin_asia is 0.044515252035678216
The coefficient for origin_europe is 0.04834854953945382
The intercept is 0.0192841161036397
```

2.b Regularized Ridge Regression

```
In [40]: ridge_model = Ridge(alpha = 0.3)
ridge_model.fit(X_train, y_train)
```

Out[40]:

Ridge		
▼ Parameters		
	alpha	0.3
	fit_intercept	True
	copy_X	True
	max_iter	None
	tol	0.0001
	solver	'auto'
	positive	False
	random_state	None

```
In [41]: print('Ridge model coef: {}'.format(ridge_model.coef_))  
#As the data has 10 columns hence 10 coefficients appear here
```

```
Ridge model coef: [ 0.31649043  0.31320707 -0.22876025 -0.70109447  0.01295851  0.37447352  
 0.37725608 -0.07423624  0.04441039  0.04784031]
```

2.c Regularized Lasso Regression

```
In [42]: lasso_model = Lasso(alpha = 0.1)  
lasso_model.fit(x_train, y_train)  
  
print('Lasso model coef: {}'.format(lasso_model.coef_))
```

```
Lasso model coef: [-0.138241      -0.01278489 -0.01642647  0.01690287 -0.51890013  0.28138241  
 0.1278489  -0.01642647  0.01690287  0.51890013  0.28138241]
```

3. Score Comparison

```
In [43]: #Simple Linear Model  
print(regression_model.score(x_train, y_train))  
print(regression_model.score(x_test, y_test))  
  
print('*****')  
#Ridge  
print(ridge_model.score(x_train, y_train))  
print(ridge_model.score(x_test, y_test))  
  
print('*****')  
#Lasso
```

```
print(lasso_model.score(x_train, y_train))
print(lasso_model.score(x_test, y_test))
```

```
0.8343770256960538
0.8513421387780067
*****
0.8343617931312616
0.8518882171608508
*****
0.7938010766228453
0.8375229615977083
```

4. Model Parameter Tuning

```
In [44]: data_train_test = pd.concat([X_train, y_train], axis =1)
data_train_test.head()
```

```
Out[44]:
```

	cyl	disp	hp	wt	acc	yr	car_type	origin_ame
350	-0.856321	-0.849116	-1.081977	-0.893172	-0.242570	1.351199	0.941412	0.773
59	-0.856321	-0.925936	-1.317736	-0.847061	2.879909	-1.085858	0.941412	-1.292
120	-0.856321	-0.695475	0.201600	-0.121101	-0.024722	-0.815074	0.941412	-1.292
12	1.498191	1.983643	1.197027	0.934732	-2.203196	-1.627426	-1.062235	0.773
349	-0.856321	-0.983552	-0.951000	-1.165111	0.156817	1.351199	0.941412	-1.292

```
In [45]: import statsmodels.formula.api as smf
ols1 = smf.ols(formula = 'mpg ~ cyl+disp+hp+wt+acc+yr+car_type+origin_america+origi
ols1.params
```

```
Out[45]: Intercept          0.019284
cyl              0.321022
disp             0.324834
hp              -0.229170
wt              -0.711210
acc              0.014714
yr               0.375581
car_type         0.381477
origin_america   -0.074722
origin_europe    0.048349
origin_asia      0.044515
dtype: float64
```

```
In [46]: print(ols1.summary())
```

OLS Regression Results

=====						
Dep. Variable:	mpg	R-squared:	0.834			
Model:	OLS	Adj. R-squared:	0.829			
Method:	Least Squares	F-statistic:	150.0			
Date:	Thu, 21 Aug 2025	Prob (F-statistic):	3.12e-99			
Time:	14:51:48	Log-Likelihood:	-146.89			
No. Observations:	278	AIC:	313.8			
Df Residuals:	268	BIC:	350.1			
Df Model:	9					
Covariance Type:	nonrobust					
=====						
	coef	std err	t	P> t	[0.025	0.975]

Intercept	0.0193	0.025	0.765	0.445	-0.030	0.069
cyl	0.3210	0.112	2.856	0.005	0.100	0.542
disp	0.3248	0.128	2.544	0.012	0.073	0.576
hp	-0.2292	0.079	-2.915	0.004	-0.384	-0.074
wt	-0.7112	0.088	-8.118	0.000	-0.884	-0.539
acc	0.0147	0.039	0.373	0.709	-0.063	0.092
yr	0.3756	0.029	13.088	0.000	0.319	0.432
car_type	0.3815	0.067	5.728	0.000	0.250	0.513
origin_america	-0.0747	0.020	-3.723	0.000	-0.114	-0.035
origin_europe	0.0483	0.021	2.270	0.024	0.006	0.090
origin_asia	0.0445	0.020	2.175	0.031	0.004	0.085
=====						
Omnibus:	22.678	Durbin-Watson:	2.105			
Prob(Omnibus):	0.000	Jarque-Bera (JB):	36.139			
Skew:	0.513	Prob(JB):	1.42e-08			
Kurtosis:	4.438	Cond. No.	7.98e+15			
=====						

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The smallest eigenvalue is 2.45e-29. This might indicate that there are strong multicollinearity problems or that the design matrix is singular.

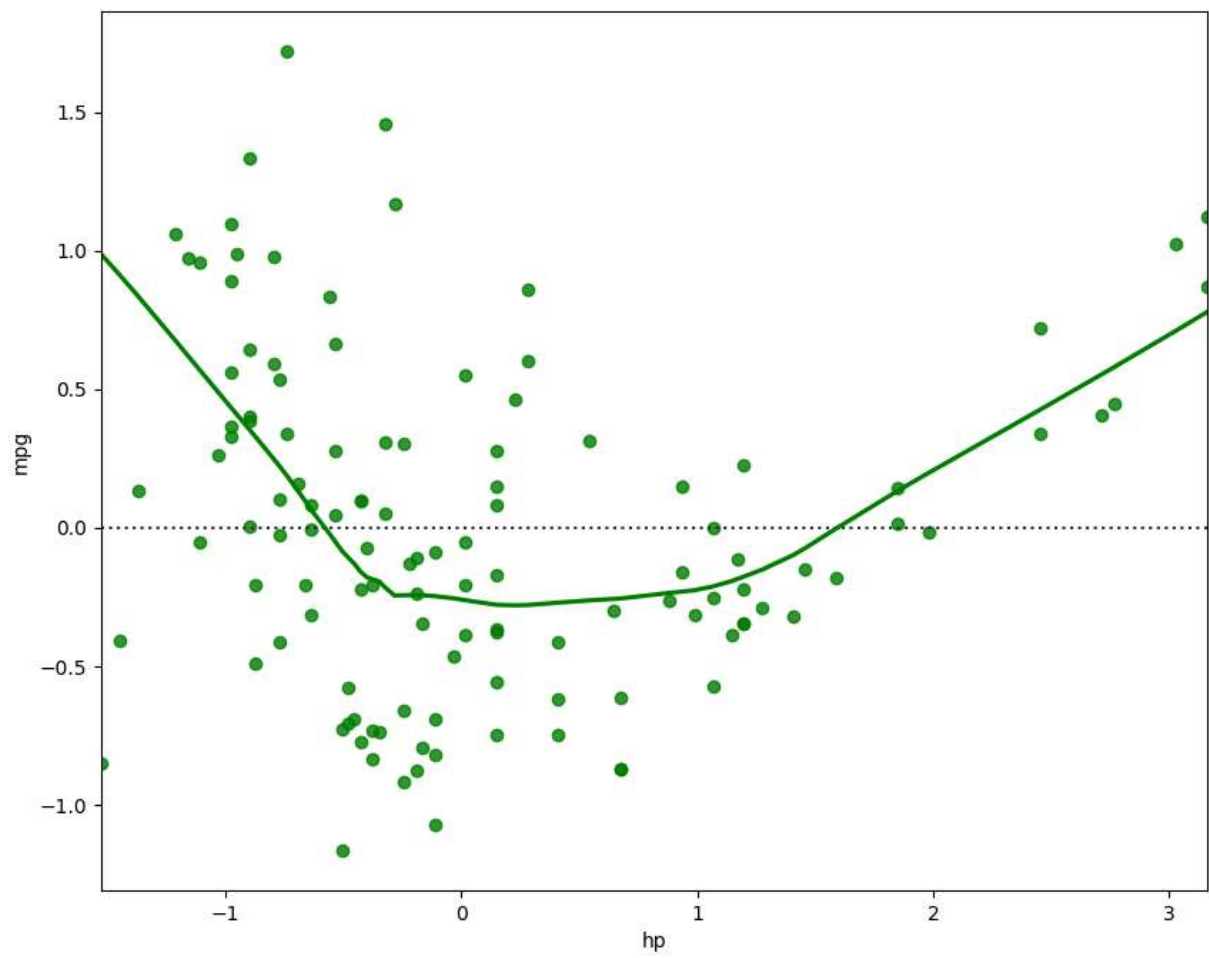
```
In [48]: mse = np.mean((regression_model.predict(x_test)-y_test)**2)

import math
rmse = math.sqrt(mse)
print('Root Mean Squared Error: {}'.format(rmse))
```

Root Mean Squared Error: 0.3776693425408783

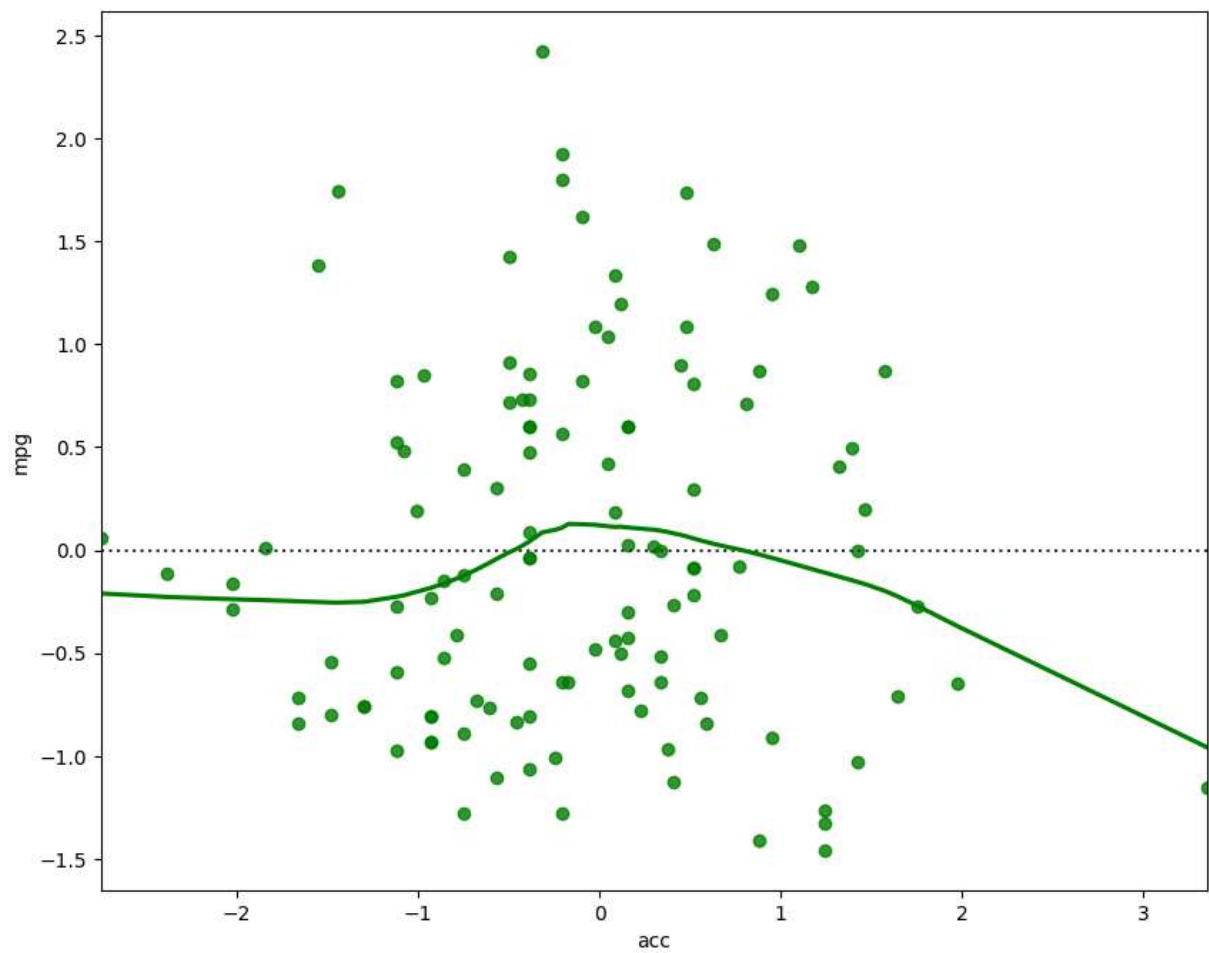
```
In [50]: fig = plt.figure(figsize=(10,8))
sns.residplot(x= x_test['hp'], y= y_test['mpg'], color='green', lowess=True )
plt.show()
```

<Figure size 1000x800 with 0 Axes>



```
In [52]: fig = plt.figure(figsize=(10,8))
sns.residplot(x= x_test['acc'], y= y_test['mpg'], color='green', lowess=True )
plt.show()
```

<Figure size 1000x800 with 0 Axes>



```
In [55]: y_pred = regression_model.predict(x_test)
plt.scatter(y_test['mpg'], y_pred)
plt.show()
```